Program Analysis Tools

Program analysis tool are for automated analysis of the source code or the executable code to produces reports regarding several important characteristics.

S. S. Satapathy

Program Analysis Tools

- A **program analysis tool** means an **automated tool** that takes the **source code or the executable code** of a program as **input** and **produces reports** **regarding several important characteristics of the program**, such as its **size**, **complexity**, **adequacy of commenting**, adherence to **programming standards**, etc. We can classify these into two broad categories of program analysis tools:
  - **Static Analysis tools**
  - **Dynamic Analysis tools**

Static program analysis tools

- **Static analysis tool** is also a **program analysis tool**.
- It assesses and **computes various characteristics of a software product** **without executing it**.
- Typically, static analysis tools analyze some structural representation of a program to arrive at certain analytical conclusions, e.g. that some structural properties hold. The structural properties that are usually analyzed are:
  - **Whether the coding standards have been adhered to**?
  - Certain **programming errors** such as **uninitialized variables** and **mismatch between actual and formal parameters**, **variables** that are **declared but never used** are also checked.
- **Code walk through** and **code inspections** **might be considered as static analysis methods**. **But**, the term static program analysis is used to denote **automated analysis tools**. So, **a** **compiler** can be considered to be **a static program analysis tool**. Other examples are **Eclipse plug-ins** widely used **for java** are **FindBug**, **PMD**. **Cppcheck** is available **for C++** programs.

Dynamic program analysis tools

- **Dynamic program analysis** techniques **require the program to be executed** and **its actual behavior recorded**.
- A dynamic analyzer usually instruments the code (i.e. **adds additional statements in the source code to collect program execution traces**).
- The instrumented code when executed allows us to record the behavior of the software for different test cases.
- After the software has been tested with its full test suite and its behavior recorded, the **dynamic analysis tool caries out a post execution analysis and produces reports** which describe the **structural coverage** that has been **achieved** by the complete test suite for the program.
- For example, the post execution dynamic analysis report might **provide data on extent statement, branch and path coverage achieved.**

Dynamic program analysis tools

- Normally the **dynamic analysis results** are **reported in the form of a histogram or a pie chart** to describe the structural coverage achieved for different modules of the program.
- The **output of a dynamic analysis tool** can be **stored** and **printed easily** and provides evidence that thorough testing has been done.
- The **dynamic analysis results the extent of testing performed in white-box mode**.
- **If the testing coverage is not satisfactory more test cases can be designed and added to the test suite**.
- Further, dynamic analysis results can help to **eliminate redundant test cases** from the test suite.

Debugging

Debugging

- Definition:
  - **Identifying errors in a program code and then fix them up**
- Debugging **approaches**
  - **Brute Force Method**
  - **Backtracking**
  - **Cause Elimination Method**
  - **Program Slicing**

Brute Force Method

- In this approach, **the program is loaded with print statements** to print the **intermediate values** with the hope that some of the printed values will help to identify the statement in error.
- This is the **Most common method** of debugging but is the **least efficient** method.
- This approach becomes more systematic with the use of a **symbolic debugger** (also called a **source code debugger**), because values of different **variables can be easily checked** and **break points** and **watch points** can be easily set to test the values of variables effortlessly.

Backtracking

- This is also a fairly common approach.
- In this approach, **beginning from the statement at which an error symptom has been observed**, the **source code is traced backwards until the error is discovered**.
- However, **as the number of source lines to be traced back increases, the number of potential backward paths increases and may become unmanageably large** thus limiting the use of this approach.

Program Slicing

- This technique is **similar** to **back tracking**.
- Here the **search space is reduced by defining slices**.
- A **slice of a program** **for** **a particular variable** **at a particular statement** **is the set of source lines preceding this statement** that can influence the value of that variable. May be similar to **du path analysis**.

Cause Elimination Method

- In this approach, a **list of causes** which could possibly have **contributed to the error symptom** is developed and tests are conducted to eliminate each.
- A related technique of identification of the error from the error symptom is the **software fault tree analysis,** for example, **Windows OS troubleshoot**.

Debugging

- Coding standards specified helps Debugging
  - Ex. Avoiding global variables, declaring variable as constants limit scope of use and help reducing search space during debugging
- Programming language paradigm also supports Debugging
  - Program written in C++ is better than that written in C